

SENSITIVITY ANALYSIS OF REWARD WEIGHTS IN MULTI-AGENT LEARNING: A BOX-PUSHING CASE STUDY

Hao Ji^{1,*}, Julian Jiang², Chuanhui Hu³, Bingling Huang⁴

¹Center for Advanced Research Computing, University of Southern California, haoji@usc.edu

²Dornsife College of Letters, Arts and Sciences, University of Southern California, julianji@usc.edu

³Department of Mechanical Engineering, University of Southern California, huchuanhui729@gmail.com

⁴Department of Mechanical Engineering, California State University, Fullerton, bihuang@fullerton.edu

ABSTRACT

Self-Organizing System consists of multiple agents that work collaboratively to accomplish complex tasks. In our prior research, we applied deep reinforcement learning (DRL) to train decentralized agents for adaptive box-pushing tasks. However, as there are different kinds of reward functions during training and each reward has its corresponding weight, finding optimal weights for the reward become crucial. Therefore, in this paper, we examined how different configurations for reward functions influence learning stability, task efficiency, and robustness. Additionally, we analyzed greedy and semi-greedy action selection strategies to balance optimality and adaptability. Experimental results demonstrate that specific reward weight settings significantly impact task success rates and resilience to external disturbances. This study contributes to advancing reinforcement learning for decentralized coordination and adaptive task execution in multi-agent systems.

Keywords: multi-agent reinforcement learning, deep Q-learning, self-organizing systems, task coordination, box-pushing problem, artificial intelligence, decentralized control

1. INTRODUCTION

In self-organizing systems (SOS), decentralized agents interact through various mechanisms to achieve complex, coordinated behaviors without centralized control. The development of SOS adopts a bottom-up paradigm because the emergent complexity emerges from the interactions between local agents [1, 2]. Adopting SOS principles for system design provides multiple benefits such as scalability and adaptability along with enhanced reliability [3, 4]. Self-organizing systems demonstrate superior resilience against environmental changes and structural break-

downs compared to traditional engineering systems, which are dependent on centralized control mechanisms [5–7].

Several different methodologies have been developed to make the SOS design more efficient. Field-based behavior regulation (FBR) [8] models the task environment through a field function, which controls the agent behavior according to their spatial positions. Field transformations enable agents to move toward areas of higher or lower field values based on predefined objectives. This methodology stands out because it operates with ease since agents need very little previous information to function because they move according to field gradients. However, the simplicity of field-based representations makes them unsuitable for complex domains because they struggle to encompass all vital task features and ignore interagent dynamics.

Researchers have investigated evolutionary design strategies [4] along with social structuring methods [5] as solutions to enhance SOS design through parametric and optimizable components. These methods integrate task fields and social fields that allow agents to use predefined social rules to support cooperation while reducing conflict and increasing task execution effectiveness [5]. The fundamental issue with social rule-based frameworks is that they depend on established interaction protocols. However, manual rule design in complex environments proves impractical because it requires complete understanding of all potential task scenarios and agent interactions prior to start.

In our prior work, multi-agent reinforcement learning (MARL) is employed to encode self-organizing knowledge for agent behavior regulation in SOS design [9–11]. However, there can be multiple reward functions in the SOS design and selecting inappropriate reward weights can lead to sub-optimal system performance. Therefore, choosing the right combination of reward weight becomes crucial in order to achieve best performance of SOS.

*Corresponding author

In this paper, we included a systematic design of experiments to evaluate how reward function weights affect SOS performance. We mainly want to address two key research questions: Q1: How does the reward weight configurations affect the system performance of self-organizing systems? Q2: How do different execution strategies (greedy vs. semi-greedy) influence the robustness of learned policies across varying reward configurations?

The remainder of this paper is structured as follows: Section 2 analyzes previous research related to self-organizing systems and reinforcement learning methodologies. Section 3 describes use of multi-agent reinforcement learning for complex SOS design. Section 4 presents a case study on the box-pushing problem, investigating how reward function weight configurations influence task efficiency and policy stability. Section 5 analyzes experimental results in depth, examining how adjustments in weight parameters affect movement patterns, success rates, and system performance across different execution strategies. Section 6 summarizes the principal conclusions from the case study and outlines future research directions.

2. RELATED WORK

2.1. Design of Complex Systems

Complex systems consist of multiple interacting elements whose combined behavior results from their complex interconnections, which complicates prediction and modeling efforts [12]. Extensive research has focused on discovering successful techniques for the development of complex systems. For example, Arroyo et al. developed a fault-adaptive framework for complex engineering environments which utilizes bio-inspired principles and binary-tree structures [13]. The study by Königseder and Shea identified that various rule application methods used in computational design synthesis show differing levels of effectiveness when applied to specific tasks like gearbox design and bicycle frame synthesis [14].

Meluso and Austin-Breneman created an agent-based simulation model for studying parameter estimation strategies within large-scale complex engineering systems [15]. McComb et al. presented a computational framework which investigates how various configuration design problem traits impact optimal team setups with factors like team size and interaction frequency [16]. Min et al. carried out research to investigate the structural complexity within engineering systems through various decomposition stages [17].

The methodology introduced by Ferguson and Lewis for creating effective reconfigurable systems like vehicles examines the evolution of system design variables alongside stability assessment through state-feedback control [18]. The design-for-variety framework by Martin and Ishii allows organizations to quickly adjust their offerings for generational shifts which helps to reduce delays in product market entry [19].

Complex system design approaches currently rely heavily on extensive domain knowledge, which turns model construction into a labor-intensive task specific to particular domains. The development of local interaction rules from overarching design requirements continues to be a primary challenge in this discipline.

Recent studies have explored reinforcement learning as a design tool for orchestrating resource flows and solving modular decision-making in system-of-systems (SoS) environments. For instance, Chen and Heydari [20] proposed the SoS Conductor framework, which uses a two-level RL architecture to coordinate subsystem actions with global resource governance. Gadi et al. [21] introduced a solver-aware system architecting (SASA) framework, where Q-learning helps identify optimal solver-module assignments based on tradeoffs between performance and cost. While these works emphasize reinforcement learning's ability to improve SoS-level design decisions, they focus primarily on *architectural and allocation heuristics* rather than the *sensitivity of reward structures themselves*, which is the subject of our study.

2.2. Artificial Self-Organizing Systems

Artificial self-organizing systems are human-engineered systems that exhibit emergent behavior and adaptability similar to natural self-organizing processes [1]. Extensive research has been conducted on the design of artificial self-organizing systems. Werfel introduced a framework in which homogeneous robots collaboratively construct predefined structures using square bricks [22]. Beckers et al. developed a robotic system where autonomous agents perform a gathering task by patrolling a designated area to collect pucks [23].

Khani and Jin proposed a social rule-based regulatory mechanism to enforce agent self-organization, enabling teams of agents to coordinate in pushing a box toward a target area [5, 6]. UAV swarms have also demonstrated self-organizing capabilities by following predefined cooperation rules to accomplish tasks such as target detection, collaborative patrolling, and formation control [24–27].

Chen and Jin applied a field-based behavior regulation (FBR) approach to guide self-organizing agents in executing complex tasks, including reaching distant targets while avoiding obstacles [8]. Price and Lamont explored the use of genetic algorithms to optimize the behavior of self-organizing unmanned aerial vehicle (UAV) swarms [28].

These robotic implementations have demonstrated the feasibility of constructing artificial self-organizing systems. Existing approaches to self-organizing system design [5, 6, 8] present promising results but suffer from inherent limitations which Section 1 details.

It is widely recognized that the behavior of artificial self-organizing systems is shaped by the coordination or reward mechanisms they are built upon. Recent studies in design automation and system architecting have further emphasized that the structure of reward functions plays a crucial role in determining system-level outcomes [20, 21, 29].

2.3. Multiagent Reinforcement Learning

While multiagent reinforcement learning (MARL) has been widely applied in cooperative control and behavioral optimization, recent work has extended its application to system-level design and architecture optimization. For example, Chen et al. [29] used meta-RL with modular policies to improve the adaptability

of collaborative robotic systems in dynamic Industry 4.0 environments. Similarly, Gadi et al. [21] explored solver assignment strategies using Q-learning to derive system-level heuristics in architecting tasks. Existing research has not yet investigated how **variations in reward configurations themselves** affect system-level dynamics. This paper aims to fill this gap.

Traditional single-agent reinforcement learning methods such as Q-learning and actor-critic can extend to multiagent reinforcement learning (RL) for environments with multiple interacting agents [30, 31]. In multiagent learning systems, multiple agents update their policies at the same time, which creates a non-stationary environment unlike the static environment of single-agent RL. The recent transition from tabular-based methods to deep reinforcement learning (DRL) developed from the requirement to manage high-dimensional state and action spaces while ensuring accurate value function approximation [32–34]. The main categories for multiagent systems include cooperative systems, competitive systems, and systems that combine both cooperative and competitive elements [32, 35–37]. The research examines cooperative agents because these agents work together towards a unified objective when designing self-organizing systems (SOS).

Optimizing individual policies or value functions stands as a common approach in multiagent reinforcement learning. Independent Q-learning represents a commonly utilized value function-based technique that enables each agent to learn its distinct state-action values through Q-learning [38, 39]. Independent Q-learning (IQL) is particularly suitable for decentralized multi-agent systems where explicit inter-agent communication is impractical or unavailable. Typical applications include cooperative robotic systems, swarm behaviors, and distributed sensor networks, where agents must rely on local observations and implicit coordination via environmental feedback. The simplicity and scalability of IQL make it advantageous in scenarios demanding robustness to environmental changes or limited communication bandwidth. Tampuu et al. integrated deep neural networks into Q-learning and implemented deep Q-networks (DQN) to train independent agents within the Pong game [32, 40]. Foerster et al. proposed the Counterfactual Multiagent (COMA) framework that uses centralized critics to assess decentralized actors during multiagent policy optimization in StarCraft [33]. Foerster et al. studied different replay stabilization methods for independent Q-learning applied to StarCraft battle scenarios [41].

Ji and Jin have applied independent Q-learning to self-organizing multi-agent systems, demonstrating its potential in regulating agent behaviors. Their findings analyze learning stability, reward sensitivity, and scalability, highlighting key factors that influence the effectiveness of decentralized coordination [9, 10].

To address the partially observable characteristics of multiagent environments, Hausknecht and Stone incorporated deep recurrent networks such as long short-term memory (LSTM) [42] and gated recurrent units (GRU) [43] for enhanced learning over extended time periods [44]. Lowe et al. developed the Multiagent Deep Deterministic Policy Gradient (MADDPG) framework which merges centralized training methods with decentralized execution for effective outcomes in predator-prey and cooperative

navigation tasks [45]. Brown and Sandholm created "Pluribus," an advanced AI system that was trained through self-play with search algorithms and demonstrated top-tier performance against expert human poker players in six-player no-limit Texas Hold'em games [46]. The research by Baker et al. showed how the combination of multiagent competition with straightforward game mechanics and extensive reinforcement learning led to the formation of self-supervised curricula, which produced strategic behaviors in hide-and-seek games [47]. Wu et al. created Bayesian delegation as a decentralized algorithm that allows agents to understand others' intentions through its application in "Overcooked," a cooperative cooking simulation [48].

Most multiagent RL approaches aim to optimize global rewards or achieve stable convergence properties. However, existing research do not fully explore the sensitivity of reward weights on system performance. Thus, this paper aims to develop computational studies that evaluate how the changes of reward function weights affect SOS performance. Insights from the study can facilitate the development of more robust and efficient self-organizing systems.

3. A DEEP MULTIAGENT REINFORCEMENT LEARNING MODEL

3.1. Single-Agent Reinforcement Learning

Understanding single-agent reinforcement learning (RL) is essential before delving into multiagent RL, as many of the foundational concepts and techniques in multiagent RL stem from single-agent learning.

A single-agent RL system trains to achieve optimal system performance by teaching an agent to independently learn task solutions through raw sensory input interpretation and environmental feedback reception. The learning framework employs the Markov Decision Process (MDP) structure which can be defined by the tuple $\langle S, A, P, R, \gamma \rangle$. The symbol S stands for the state space which includes every potential environmental condition that an agent might face while A indicates all possible actions that the agent can take. The transition matrix P expresses the probabilistic transitions between states while the reward function R incorporates a discount factor γ to give priority to immediate rewards over distant rewards. The agent strives to achieve the highest cumulative discounted reward which is expressed by $R_t = \sum_{t=0}^T \gamma^t r_t$ where T stands for when the task finishes. The agent calculates the best action-value function $Q(s, a)$ through the application of the Bellman equation.

$$Q_{\pi}(s, a) = E[r + \gamma \max_{a'} Q(s', a') | s, a] \quad (1)$$

This equation iteratively updates the Q-value function until convergence [30].

Q-learning has found extensive application in single-agent reinforcement learning according to historical research evidence [49, 50]. The method uses Q-tables to remember each state-action pair in a table format. Q-table-based learning systems perform well with discrete state-action spaces but fail to scale for real-world applications that feature continuous or high-dimensional state-action spaces [49, 50].

Function approximators like deep neural networks have been merged with Q-learning to estimate Q-values which resulted in the development of deep Q-learning [40]. Deep Q-learning involves training a neural network with parameters θ_i by minimizing its loss function during each iteration i :

$$L_i(\theta_i) = E[(y_i - Q(s, a; \theta_i))^2] \quad (2)$$

where the target value for iteration i is:

$$y_i = E\left[r + \gamma \max_{a'} Q(s', a'; \theta_{i-1})\right] \quad (3)$$

The gradient of this loss function is computed as:

$$\nabla_{\theta} L_i(\theta_i) = E_{s,a,r,s'} \left[(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)) \nabla_{\theta} Q(s, a; \theta_i) \right] \quad (4)$$

3.2. Multiagent Reinforcement Learning

The previous discussion highlighted two main approaches that exist for training multiple agents within reinforcement learning systems. One method considers all agents as one unit which allows them to learn together as a single team. While this method demonstrates effective convergence comparable to single-agent reinforcement learning techniques, it faces substantial limitations in terms of scalability. The study uses multiagent independent deep Q-learning (IQL) to enhance learning efficiency while achieving scalability. The framework assigns a discrete action space A_i to each agent i from 1 to n resulting in the formation of a collective action domain $A = A_1 \times A_2 \times \dots \times A_n$. Each agent functions in an identical state space and earns rewards through a common reward function.

The IQL approach requires each agent to operate with its own neural network which is trained via deep Q-learning and experience replay. Agents perceive the state space individually and update their respective policy and value functions. Action selection is based on each agent's learned network, which derives rewards from the environment via the shared reward function. The identical state-action space that all agents share enables the learned networks to work with various team sizes thus enabling multiagent system scalability. Due to the minimal or nonexistent explicit communication between agents, independent deep Q-learning is particularly suitable for training self-organizing agents. Other multiagent reinforcement learning techniques, such as deep distributed recurrent Q-learning [51], adds communication between agents which extends training time but isn't needed for self-organizing agent systems. Building on these advantages of independent Q-learning, this study applies IQL in a multi-agent box-pushing environment to examine the impact of different reward function weight configurations on system performance.

In this study, we apply independent Q-learning (IQL) in a multi-agent box-pushing environment with a fixed team of five agents to investigate how different reward function weight configurations influence system performance. Each agent follows a decentralized learning process, updating its policy based on its own experiences while interacting within a shared environment. Unlike centralized learning approaches, our framework

maintains scalability by enabling agents to make independent decisions without explicit communication.

We assess reward function design effects by systematically changing the weights of goal reward (w_1), collision reward (w_2), rotation reward (w_3), and distance reward (w_4) throughout a specified parameter space. This study investigates the effects of various configurations on movement efficiency and policy stability as well as task success rates. Our research explores whether extreme weight values in reward functions cause undesirable agent behaviors, such as excessive movement, inefficient rotations, and overly cautious decision-making. To assess robustness under different reward configurations, we compare greedy and semi-greedy execution strategies in our experiments.

4. CASE STUDY

To analyze how reward weight configurations influence policy stability and task efficiency, we conducted a case study on the box-pushing problem using independent Q-learning with 5 agents.

4.1. The Box-Pushing Problem

The box-pushing problem is commonly classified as a trajectory planning or piano mover's problem [52]. Various topological and numerical methodologies have been proposed to address this problem [52]. In this study, we employ a decentralized, self-organizing multiagent deep Q-learning framework to address the box-pushing challenge. During the training phase, each agent learns policies through independent Q-learning (IQL), and coordinate to push the box towards a goal state without explicit system-wide control.

For experimental implementation, we utilize pygame, a Python-based multiagent game development framework. In the simulated box-pushing environment, each agent is trained with an independent Q-learning (IQL) neural network. The trained networks are later evaluated under test conditions. Our analysis investigates the behavioral properties, consistency, and robustness of the acquired policies under varying reward configurations.

A graphical representation of the box-pushing scenario is illustrated in Fig. 1. The simulation environment features a game screen measuring 600 pixels in width and 480 pixels in height. The agents, represented as green squares, have constrained sensing and movement capabilities, requiring them to cooperatively push and rotate the box (shown as a brown rectangle, depending on its rotation state). The goal is to move the box towards a target location marked by a "+" symbol.

Obstacles, such as red circular barriers and thick white boundary walls, introduce complexity by restricting the agents' movement. While agents can only push the box and cannot directly rotate it, they must collaboratively generate rotational forces through coordinated pushing [5, 6]. The environment includes boundary sensors positioned outside the box to detect its interactions with obstacles.

The simulation operates in a pixel-based grid environment, where each pixel represents a discrete spatial unit. The box size is 60 pixels wide, 150 pixels long, as depicted in Fig. 1.

The agents in the box-pushing task exhibit *limited sensing and communication capabilities*. Each agent possesses a sensor

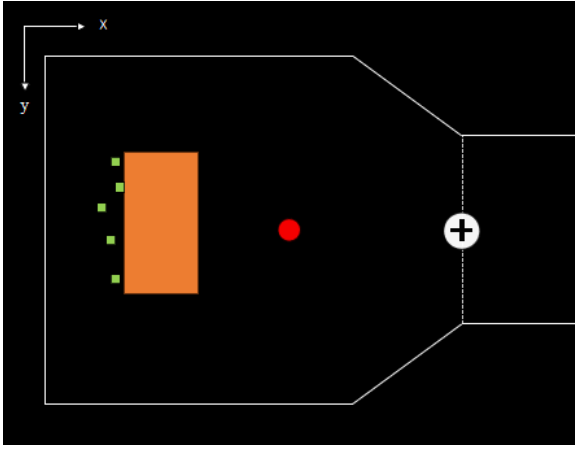


FIGURE 1: A GRAPHICAL REPRESENTATION OF THE BOX-PUSHING SCENARIO.

that detects the orientation of the box and measures its distance to nearby obstacles. However, agents experience limited visual awareness as they obtain partial information about the current state along with reward data and subsequent state transitions. To achieve effective actions from their limited sensory data, agents must use neural networks to process these perceptual inputs. The underlying assumptions correspond to minimalist robotic design principles [53] while reflecting actual physical robot systems [54, 55].

4.2. State Space and Action Space

4.2.1. Task State Space The state space of the box-pushing task is illustrated in Fig. 2. The sensor located at the box's center can identify any surrounding obstacles in order to gather essential environmental information. The sensor covers a circular area with a radius of 150 pixels, divided into eight equal segments, as illustrated in Fig. 2. The state representation at each moment reflects sector occupancy status, with '1' indicating the presence of an obstruction and '0' showing that the sector is unoccupied. The sensor in the system also provides readings about the box's x-axis orientation compared to the target location [49, 50].

By denoting the sectors as $s_1, s_2, \dots, s_8$, and the box orientation as s_9 , the overall state space is defined as:

$$S = \{s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9\} \quad (5)$$

For instance, in Fig. 2, sectors s_3, s_5 , and s_7 are obstructed by obstacles, resulting in the values $s_3 = 1, s_5 = 1, s_7 = 1$, while the remaining sectors have 0. The box's orientation relative to the x-axis is approximately 30 degrees, which can be normalized to the range $[-1, 1]$ using the transformation $s_9 = \theta/180 = 0.83$. This normalization method is widely utilized in deep Q-network training and reinforcement learning applications [49, 50].

Accordingly, the state representation can be expressed as a nine-element vector:

$$\langle 0, 0, 1, 0, 1, 0, 1, 0, -0.83 \rangle \quad (6)$$

During training, an agent is positioned near the box's center to receive locally broadcast sensor data. The sensor also provides

distance measurements from the box's center to the goal, which is similar to real-world radar applications. This aligns with previous gradient-based methodologies where task fields determine agent movements [5, 6]. Agents leverage this sensor-derived distance information for decision-making. In our implementation, agent movement refers specifically to stepwise motion along the four adjacent edges of the box (i.e., front, back, left, and right), and does not include diagonal or corner-based positions.

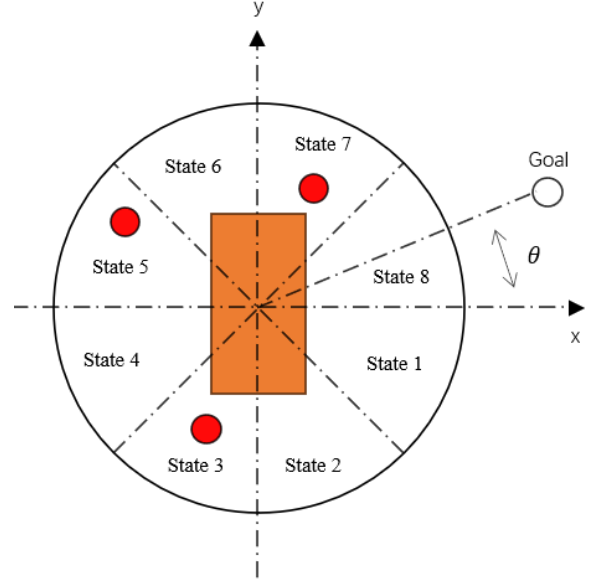


FIGURE 2: BOX-PUSHING TASK STATE REPRESENTATION.

4.2.2. Box Neighborhood The box neighborhood consists of six predefined regions [6, 56], as depicted in Fig. 3. During simulations, an agent may move to any of these six neighborhood regions, which define its position relative to the box. Multiple agents can occupy the same neighborhood region because their small size aligns with the definition of the minimalist robot system [53].

4.2.3. Box Dynamics The motion of the box is governed by a simplified physical model that incorporates force and torque effects. The pushing force exerted by agents is computed as the sum of the individual force vectors applied by each pushing agent. When agents exert force perpendicular to the box, it moves forward, while diagonal pushes generate torque, resulting in rotational movement. The box moves forward when agents apply perpendicular force to it, but generates rotational movement when agents push at diagonal angles because these pushes create torque. When two agents apply force to the opposite corners, they enable the box to rotate at a rate of 10 degrees per step. Due to inertia, the box maintains its course until it meets an obstacle that halts its movement.

4.2.4. Agent Action Space The agents' action space is defined by the box's neighborhood and its simulated motion dynamics. At each time step, an agent selects a position from one of the six designated box neighborhood regions and applies a pushing force. Consequently, the joint action space consists of:

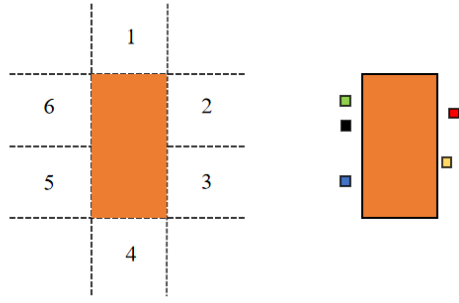


FIGURE 3: THE SIX REGIONS OF THE BOX NEIGHBORHOOD.

$$A = \{a_1, a_2, a_3, a_4, a_5, a_6\} \quad (7)$$

For instance, if an agent chooses action a_1 , it relocates to neighborhood region “1” and pushes the box from that position. The box responds by moving forward or rotating, depending on the applied forces. Similar principles apply to other actions. The box’s movement requires multiple coordinated pushes across time steps, with the number of steps needed depending on the starting and target positions.

4.3. Reward Schema and Training Model

To effectively train multiple agents for self-organization in pushing the box to the designated goal area, an appropriate reward structure must be established to guide agent learning. Building upon previous Q-table-based reward models for box-pushing [49, 50], we developed a novel reward framework tailored for training agents in the box-pushing task.

The overall reward function consists of four key components: *distance*, *rotation*, *collision*, and *goal*.

4.3.1. Distance Reward The reward function for minimizing the distance between the box and the target position is denoted as R_{dis} and formulated in Eq. (8). The previous distance, D_{old} , represents the measured pixel distance between the box and the goal in the previous time step, while D_{new} denotes the updated distance at the current step. A scaling factor C_d , known as the *distance coefficient*, is introduced in our simulation and set to 2.5. At each time step, agents compute the difference between D_{old} and D_{new} and determine the corresponding distance reward using the following equation:

$$R_{dis} = (D_{old} - D_{new}) * C_d \quad (8)$$

4.3.2. Rotation Reward The rotation reward R_{rot} is defined in Eq. (9), where a_1 and a_2 represent the previous and current angular positions of the box’s x -axis relative to the goal position. This reward is designed to discourage abrupt rotations exceeding 11 degrees. By limiting excessive rotation, the agents ensure smooth movement and avoid strong rotational momentum, which could lead to collisions with obstacles. As the primary objective is to push the box toward the goal rather than excessive rotation, the rotation reward is utilized conservatively:

$$R_{rot} = \cos(a_2 - a_1) - 0.98 \quad (9)$$

4.3.3. Collision Reward The collision reward mechanism follows standard collision avoidance strategies [56] and is expressed in Eq. (10). During each simulation step, if the box does not collide with any obstacles or walls, the collision reward remains zero ($R_{col} = 0$). However, if a collision occurs, all agents receive a penalty of -1 , discouraging unintended contact:

$$R_{col} = \begin{cases} -1, & \text{if collision occurs} \\ 0, & \text{if no collision occurs} \end{cases} \quad (10)$$

4.3.4. Goal Reward The reward for successfully reaching the goal, denoted as R_{goal} , is given by:

$$R_{goal} = \begin{cases} 1, & \text{if reaching goal} \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

Each simulation step evaluates whether the box reaches the goal position. If the goal is reached, all agents receive a positive reward of 1, otherwise, no reward is given.

The total reward function integrates multiple components, including distance, rotation, collision, and goal rewards. The combined reward function is represented as a weighted sum:

$$R_{tot} = w_1 R_{goal} + w_2 R_{col} + w_3 R_{rot} + w_4 R_{dis} \quad (12)$$

In our simulations, we extend prior work [49, 50] by systematically evaluating different reward weight configurations. We followed the same reward configuration strategy used in Section 4.4.1 to systematically evaluate different reward weight combinations.

During training, since all agents are homogeneous and cooperatively work towards pushing the box, they receive identical rewards. This ensures that the reward functions defined in Eqs. (10) through (12) are exclusively based on the box’s position and orientation. By structuring the rewards in this manner, each agent’s neural network treats the actions of other agents as part of its environment, learning to navigate the action space effectively and optimize its policy using an ϵ -greedy action selection strategy.

Over time, agents learn to distinguish their individual actions while still coordinating effectively to push the box toward the goal. This cooperative learning mechanism is a defining characteristic of multiagent independent deep Q-learning (IQL) neural networks.

4.4. Experiment Design

To investigate the impact of different reward function weight configurations on multi-agent policy behavior, we conducted a series of training and testing experiments using the independent deep Q-learning framework. The objective was to analyze the stability of acquired policies, the effects on agent movement strategies, and the robustness of learned behaviors under varying reward settings

4.4.1. Multiagent Reinforcement Learning-Based Agent Training As illustrated in Fig. 4, the independent deep Q-learning algorithm was applied to train agents in a decentralized manner. Each agent independently learned policies using its own neural network, and coordination emerged through shared environmental interactions. The agents were homogeneous and received identical rewards to encourage cooperative behavior. The dependent variables of the training process include Cumulative Reward, which signifies the learning behavior of the agents; Agent Travel Distance, which captures variations in movement patterns among agents; Success Indicator (0 or 1), which assesses task completion; and a set of neural networks trained under different random seeds, denoted as NNs, representing the learned policies.

Unlike previous work that used fixed reward function weights [9, 10, 49], our study systematically investigated the impact of different weight combinations across four reward components: *goal*, *collision*, *rotation*, and *distance*. The corresponding experimental setup is detailed below.

- **Reward Configuration:** Each of the four reward components (w_1, w_2, w_3, w_4) was assigned one of four values {0, 1, 10, 100}, resulting in 256 unique reward configurations.
- **Number of Agents:** Fixed at five agents in each experiment.
- **Training Environment:** Identical to previous work [9, 10], including the simulation settings and physical constraints.

Each training episode was executed until one of three termination conditions were met:

1. The box reached the goal position.
2. The box collided with an obstacle or the environment boundary.
3. The training episode exceeded 500 time steps.

Each configuration was trained over 6,000 episodes to ensure stable convergence of the Q-learning process. For statistical stability, each training setup was run across 100 different random seeds. Also, the mean values of cumulative rewards were recorded.

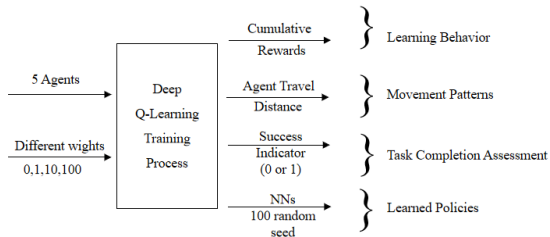


FIGURE 4: AGENT TRAINING EXPERIMENT DESIGN.

4.4.2. Testing and Performance Evaluation After training, the learned policies were evaluated under different reward configurations to assess their effectiveness, robustness, and generalization capabilities. The testing phase focused on analyzing the impact of varying reward weights on success rates under two different execution strategies: **greedy execution** and **semi-greedy execution**.

Independent Variables. The independent variables in the testing process were:

1. **Reward Configuration:** Each of the four reward components (w_1, w_2, w_3, w_4) was assigned one of four values {0, 1, 10, 100}, maintaining consistency with the training phase.
2. **Execution Strategy:**
 - (a) **Greedy Execution:** The agent strictly followed the learned policy by always selecting the action with the highest Q-value.
 - (b) **Semi-Greedy Execution:** The agent followed an ϵ -greedy policy with $\epsilon = 0.3$, allowing 30% of actions to be selected randomly to encourage exploration.

Dependent Variables. The performance of each learned policy was evaluated based on:

1. **Success Rate:** The percentage of test episodes in which the agents successfully pushed the box to the goal position without colliding with obstacles or exceeding the maximum step limit.
2. **Average Reward:** The cumulative reward obtained by the agents during testing, reflecting the efficiency of learned policies under different weight configurations.
3. **Average Movement Distance:** The mean distance traveled by each agent per episode, indicating behavioral variations induced by different reward weightings.

Testing Procedure. Each trained policy was evaluated over 100 test episodes, ensuring statistical stability. The agents started from randomized initial positions while the goal and environmental constraints remained unchanged. The success rate, average reward, and movement distance were recorded for each reward configuration under both execution strategies.

Evaluation Metrics. The following key aspects were analyzed:

1. **Effectiveness of Reward Weights:** How different weight combinations influence task performance.
2. **Policy Robustness:** How well policies perform when introducing random exploration ($\epsilon = 0.3$).
3. **Behavioral Trends:** The impact of different reward functions on agent movement patterns.

The results obtained from these evaluations provided insights into the optimal reward configurations for effective policy learning and robustness in dynamic environments.

5. RESULTS AND DISCUSSION

5.1. Reward Analysis

Figures 5–8 illustrate the impact of different weight configurations on the average reward. By fixing one weight equals to 1 (e.g., $w_1 = 1$) while varying the others, we observe how different reward components influence agent performance.

From the results, we note the following key observations:

- The experimental results show that varying w_1 , w_2 , and w_3 has a relatively minor impact on average reward. As seen in fig 5–7, increasing or decreasing these weights does not cause significant fluctuations in the reward distribution. The trends remain largely stable, indicating that goal reward (w_1), collision penalty (w_2), and rotation penalty (w_3) contribute to shaping agent behavior but do not drastically alter overall performance.
- The impact of w_4 on reward is substantial, as shown in fig 5–7, where increasing w_4 leads to a significant rise in the average reward. However, in fig 8, where w_4 is fixed at 1, shows a drastic reduction in the overall reward scale. Unlike the previous cases where reward values exceed 8000, the maximum reward in fig 8 is only around 320. This suggests that w_4 could play a crucial role in determining reward magnitude.

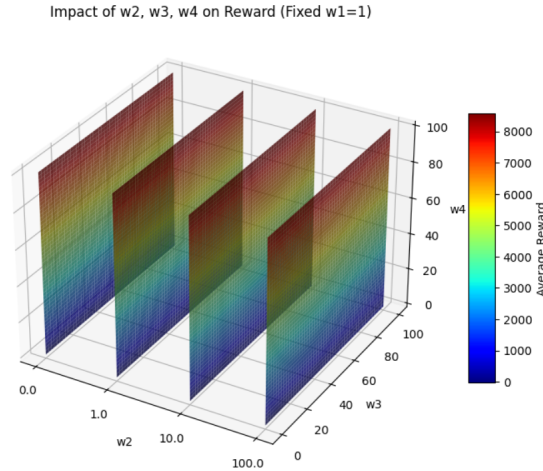


FIGURE 5: IMPACT OF w_2 , w_3 , w_4 ON REWARD (FIXED $w_1=1$)

5.2. Quality of Learned Knowledge

The effectiveness of the learned policy is evaluated during the testing phase by measuring the success rate across 100 random seeds. The success rate in a specific data point in Figure 9 is computed as the average success rate across all possible combinations of the other weights. Specifically, for each given weight setting (w_1, w_2, w_3, w_4), the success rate is averaged over $4 \times 4 \times 4 \times 100 = 6400$ independent experiments. This ensures that the reported success rate reflects the overall impact of each weight while accounting for variations in the other parameters.

1. Impact of Reward Weights on Success Rate under Greedy Execution (Figure 9)

Impact of w_1 , w_3 , w_4 on Reward (Fixed $w_2=1$)

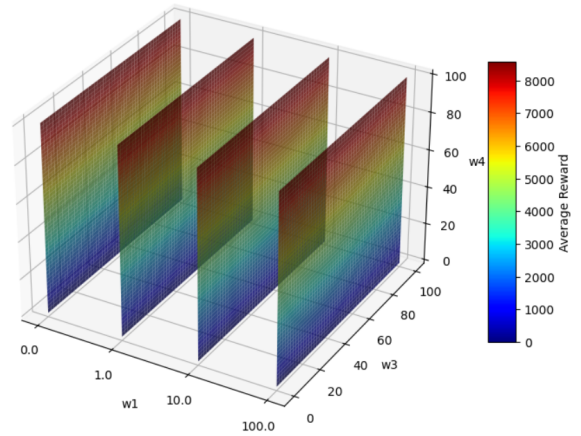


FIGURE 6: IMPACT OF w_1 , w_3 , w_4 ON REWARD (FIXED $w_2=1$)

Impact of w_1 , w_2 , w_4 on Reward (Fixed $w_3=1$)

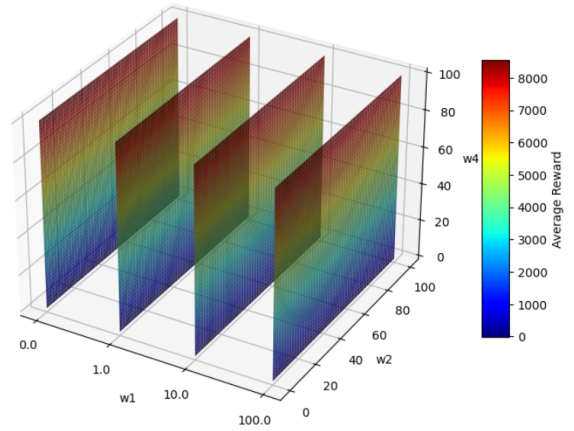


FIGURE 7: IMPACT OF w_1 , w_2 , w_4 ON REWARD (FIXED $w_3=1$)

- w_1 and w_2 exhibit relatively stable effects across different values, with moderate values leading to the best success rate.
- When w_4 is moderate (e.g., 1), success rate is maximized, suggesting that an appropriate distance incentive leads to optimal coordination.
- Excessively high w_4 (e.g., 100) significantly decreases the success rate, possibly due to agents overly prioritizing the distance coverage at the expense of necessary intermediate adjustments.

2. Impact of Reward Weights on Success Rate under Semi-Greedy Execution (Figure 10)

- The semi-greedy setting achieves slightly lower success rates compared to the purely greedy strategy, but maintains robustness across different greedy weight configurations.

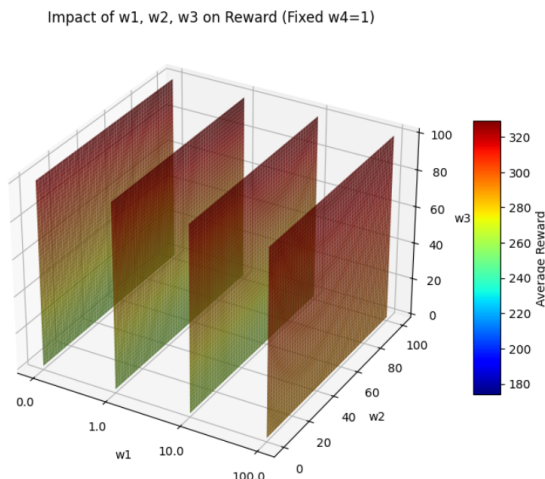


FIGURE 8: IMPACT OF w_1 , w_2 , w_3 ON REWARD (FIXED $w_4=1$)

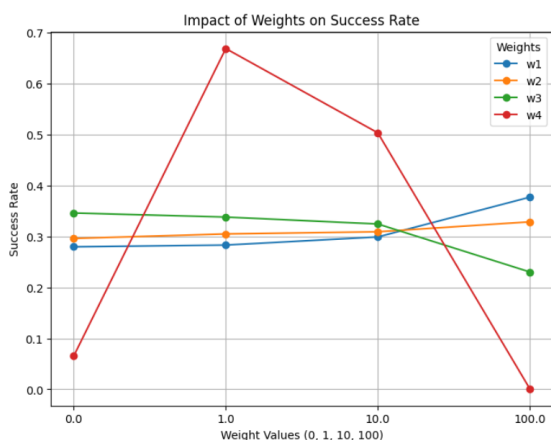


FIGURE 9: IMPACT OF WEIGHTS ON SUCCESS RATE (GREEDY)

- The trend for w_4 remains consistent with the greedy execution case, where a moderate value (10) yields the highest success rate. However, extreme values (0 or 100) may negatively impact performance.
- Excessive emphasis on a single reward component, such as distance coverage (w_4) or collision avoidance, can lead to suboptimal performance.

In summary, greedy execution concentrates on actions with the highest Q-values, which enables quick convergence yet creates extreme susceptibility to reward variations, leading to performance issues when improperly configured. Semi-greedy execution combines 30% exploration, which helps to avoid overfitting while simultaneously increasing adaptability. Experimental findings indicate that both strategies reach optimal success rates at $w_4 = 1$ or $w_4 = 10$ by maintaining a balance between coordination and movement efficiency. Greedy execution experiences severe performance drops as w_4 rises because it focuses on movement rather than correction, yet semi-greedy execution achieves stable results by using exploration to buffer against extreme reward

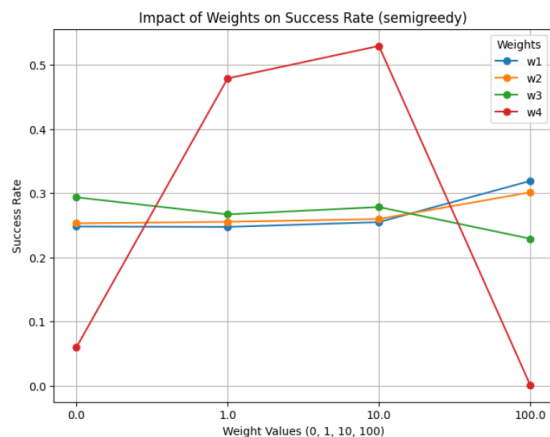


FIGURE 10: IMPACT OF WEIGHTS ON SUCCESS RATE (SEMIGREEDY)

dependencies.

5.3. Comprehensive Analysis of Reward Weights on Agent Movement Behavior

To evaluate the effectiveness of different reward configurations, we analyze the impact of w_1 (goal reward), w_2 (collision reward), w_3 (rotation reward), and w_4 (distance reward) on the average movement distance of the agent. The average distance is computed as the mean distance traveled by each agent across 6000 episodes per experiment, averaged over multiple trials for each weight setting. Specifically, for a given weight value (e.g., $w_1 = 0$), the average distance is calculated from all experimental runs where $w_1 = 0$, covering $4 \times 4 \times 4 \times 100$ data points from the 100 random seeds with all other weight combinations.

In this study, the term agent movement distance does not refer to the Euclidean distance between the agent and the goal. Instead, it represents the *stepwise movement of the agent within the predefined box neighborhood regions* (as illustrated in Fig. 3). The environment is divided into six distinct regions surrounding the box, and movement between these regions determines the recorded distance. Specifically, when an agent moves from one adjacent region to another, the distance increases by 1. If an agent moves across a non-adjacent region, the distance increases by 2. Conversely, if an agent repeats the same action in successive steps, the recorded movement distance remains 0. This hierarchical measurement framework captures how much the agent actively moves in its role during the box-pushing task rather than its proximity to the goal.

The findings of these results, in conjunction with the success rate and the trends of average rewards, provide deeper insight into task completion efficiency, learning stability, and strategy formation.

1. **Effect of Goal Reward (w_1) on Movement Behavior** The goal reward weight (w_1) has a limited effect on the agents' movement patterns. As seen in Fig. 11, increasing w_1 slightly reduces the total movement distance, indicating that agents may prioritize direct goal attainment. However, the overall change remains small. We can suggest that

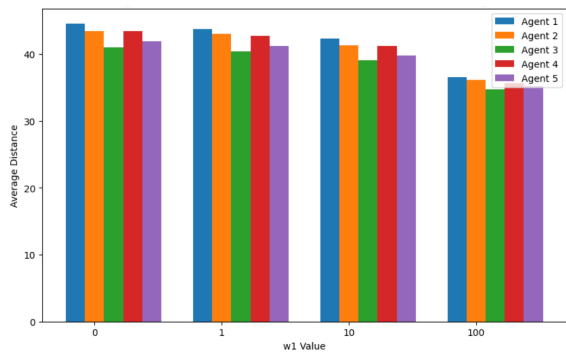


FIGURE 11: EFFECT OF w_1 ON AGENT MOVEMENT DISTANCE

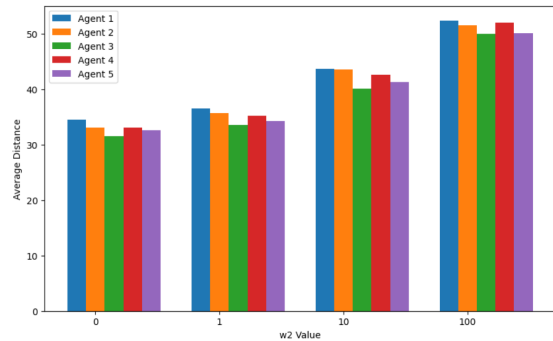


FIGURE 12: EFFECT OF w_2 ON AGENT MOVEMENT DISTANCE

goal incentives alone do not strongly influence the agents' movement behaviors. This is likely because other factors such as collision avoidance and rotation adjustments are necessary for effective navigation.

2. Effect of Collision Reward (w_2) on Movement Efficiency

In Figure 12, increasing the collision reward weight (w_2) significantly impacts movement behavior. When w_2 values increase, the total movement distances extend as agents demonstrate more cautious and evasive movement patterns. Focusing on avoiding collisions causes agents to move more exploratory, which can decrease efficiency when direct routes exist.

3. Effect of Rotation Reward (w_3) on Safe and Efficient Navigation

The rotation reward weight (w_3) influences how frequently agents adjust their orientations. Fig. 13 indicates that moderate w_3 values result in increased movement, likely due to agents optimizing their positioning before pushing the box. However, the high rotational penalty from w_3 prevents agents from moving effectively because they become excessively restricted and adopt inefficient or static strategies.

4. Effect of Distance Reward (w_4) on Task Completion and Efficiency

Figure 14 shows that The distance reward weight (w_4) has the most pronounced effect on movement behavior. When w_4 is low, agents travel extensively, possibly due to aimless exploration or inefficient task execution.

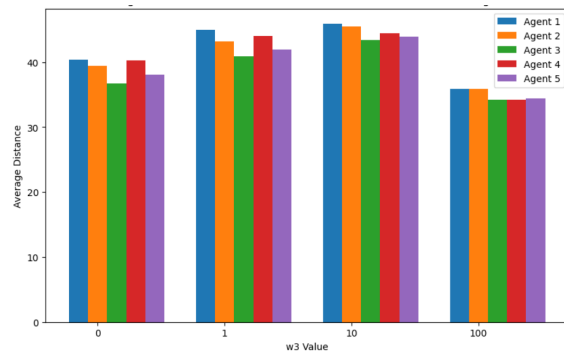


FIGURE 13: EFFECT OF w_3 ON AGENT MOVEMENT DISTANCE

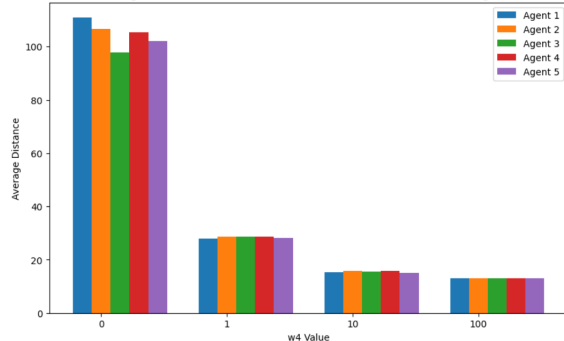


FIGURE 14: EFFECT OF w_4 ON AGENT MOVEMENT DISTANCE

As w_4 increases, movement distances sharply decrease, which indicates that agents learn to minimize unnecessary steps. However, excessively high w_4 values may lead to overly rigid strategies, where agents sacrifice adaptability for larger distance coverage.

5. Cross-Analysis with Success Rate and Learning Stability

By integrating movement distance trends with the success rate curves, we observe: When analyzing movement behaviors alongside success rates fig 9–10, the best performance is observed under balanced weight configurations. Specifically, moderate w_4 values lead to optimal task completion, while extreme values negatively affect success rates. Similarly, excessive collision or rotation penalties (w_2 and w_3) introduce instability in learning. These findings suggest that a well-tuned reward structure is critical for maintaining both movement efficiency and task success.

These findings underscores that creating well-balanced reward functions is essential for successful reinforcement learning outcomes. Strong goal incentives improve task efficiency, but focusing too much on one aspect of rewards results in unwanted behaviors including cautious movements, excessive exploration, or rigid approaches.

6. CONCLUSIONS AND FUTURE WORK

6.1. Conclusions

This study analyzed the impact of different reward weight configurations on agent movement behavior in a multi-agent box-

pushing task. By systematically varying the weights of goal reward (w_1), collision reward (w_2), rotation reward (w_3), and distance reward (w_4), we observed how agents' movement efficiency and success rate were influenced by these parameters.

Key findings from the experiments include:

- **Distance reward (w_4) has the most significant impact on movement behavior.** Agents travel greater distances when w_4 falls below the optimal level because they move excessively around the box. The increase of w_4 leads agents to minimize unnecessary movement as they focus more on distance coverage. Agents lose adaptability when the value of w_4 becomes too high because it may create rigid policies.
- **Goal reward (w_1) moderately influences movement distance.** A low w_1 leads to aimless movement patterns, while a higher w_1 encourages agents to commit to reaching the goal more directly. However, its effect is secondary to distance reward in shaping movement efficiency.
- **Collision reward (w_2) reduces movement but may limit adaptability.** A high w_2 discourages unnecessary movement and helps agents avoid obstacles. However, excessive collision penalties may lead to overly conservative strategies, which will prevent agents from taking necessary risks to complete the task.
- **Rotation reward (w_3) increases movement distance.** Higher w_3 values lead to more frequent rotation adjustments, which prolong travel distances. Optimizing w_3 is necessary to balance effective orientation changes without introducing excessive movement inefficiencies.

These results highlight the importance of designing a well-balanced reward function that optimizes both task efficiency and policy flexibility. Over-emphasizing a single reward component can lead to unintended behaviors, such as excessive exploration, inefficient movement, or over-cautious policies.

6.2. Future Work

While this study explored discrete reward values on SOS performance. Future research should focus on understanding how agents adapt their policies over time, rather than just evaluating final policy outcomes. A time-based analysis of learning stability and convergence could reveal how different reward settings influence training dynamics. Another important direction is testing model generalizability in dynamic environments, such as those with shifting goal locations or varying obstacle distributions.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support of the USC Center for Advanced Research Computing for providing the computational resources necessary to conduct these experiments.

REFERENCES

- [1] Reynolds, C. W. "Flocks, Herds and Schools: A Distributed Behavioral Model." *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*: pp. 25–34. 1987. Anaheim, CA.
- [2] Ashby, W. R. "Facets of Systems Science." Klir, G. J. (ed.). *Facets of Systems Science*. Springer, Boston, MA (1991): pp. 405–417.
- [3] Chiang, W. and Jin, Y. "Design of Cellular Self-Organizing Systems." *ASME International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Vol. 45028: pp. 511–521. 2012. American Society of Mechanical Engineers, Chicago, IL.
- [4] Humann, J., Khani, N. and Jin, Y. "Evolutionary Computational Synthesis of Self-Organizing Systems." *AI EDAM* Vol. 28 No. 3 (2014): pp. 259–275.
- [5] Khani, N. and Jin, Y. "Dynamic Structuring in Cellular Self-Organizing Systems." *Design Computing and Cognition'14*. Springer, Cham (2015): pp. 3–20.
- [6] Khani, N., Humann, J. and Jin, Y. "Effect of Social Structuring in Self-Organizing Systems." *ASME J. Mech. Des.* Vol. 138 No. 4 (2016): p. 041101.
- [7] Ji, H. and Jin, Y. "Modeling Trust in Self-Organizing Systems With Heterogeneity." *ASME 2018 International Design Engineering Technical Conferences and Computers and Information in Engineering Conferences*. 2018. Quebec City, Canada.
- [8] Chen, C. and Jin, Y. "A Behavior Based Approach to Cellular Self-Organizing Systems Design." *ASME International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Vol. 54860: pp. 95–107. 2011. Washington, DC.
- [9] Ji, Hao and Jin, Yan. "Knowledge Acquisition of Self Organizing Systems With Deep Multiagent Reinforcement Learning." *Journal of Computing and Information Science in Engineering* Vol. 22 No. 2 (2021): pp. 021010–1–021010–12. DOI [10.1115/1.4052800](https://doi.org/10.1115/1.4052800).
- [10] Ji, Hao and Jin, Yan. "Evaluating the learning and performance characteristics of self-organizing systems with different task features." *Artificial Intelligence for Engineering Design, Analysis and Manufacturing* Vol. 35 No. 4 (2021): pp. 404–422. DOI [10.1017/S089006042100024X](https://doi.org/10.1017/S089006042100024X).
- [11] Ji, Hao and Jin, Yan. "Impact of Task Constraint on Agent Team Size of Self-Organizing Systems Measured by Effective Entropy." *Journal of Computing and Information Science in Engineering* Vol. 24 No. 8 (2024).
- [12] Bar-Yam, Y. *General Features of Complex Systems*. Vol. 1. UNESCO, EOLSS Publishers, Oxford, UK (2002).
- [13] Arroyo, M., Huisman, N. and Jensen, D. C. "Exploring Natural Strategies for Bio-Inspired Fault Adaptive Systems Design." *ASME J. Mech. Des.* Vol. 140 No. 9 (2018): p. 091101.
- [14] Königseder, C. and Shea, K. "Comparing Strategies for Topologic and Parametric Rule Application in Automated Computational Design Synthesis." *ASME J. Mech. Des.* Vol. 138 No. 1 (2016): p. 011102.
- [15] Meluso, J. and Austin-Breneman, J. "Gaming the System: An Agent-Based Model of Estimation Strategies and Their Effects on System Performance." *ASME J. Mech. Des.* Vol. 140 No. 12 (2018): p. 121101.
- [16] McComb, C., Cagan, J. and Kotovsky, K. "Optimizing Design Teams Based on Problem Properties: Computational

- Team Simulations and an Applied Empirical Test.” *ASME J. Mech. Des.* Vol. 139 No. 4 (2017): p. 041101.
- [17] Min, G., Suh, E. S. and Hölttä-Otto, K. “System Architecture, Level of Decomposition, and Structural Complexity: Analysis and Observations.” *ASME J. Mech. Des.* Vol. 138 No. 2 (2016): p. 021102.
- [18] Ferguson, S. M. and Lewis, K. “Effective Development of Reconfigurable Systems Using Linear State-Feedback Control.” *AIAA J.* Vol. 44 No. 4 (2006): pp. 868–878.
- [19] Martin, M. V. and Ishii, K. “Design for Variety: Development of Complexity Indices and Design Charts.” *ASME Design Engineering Technical Conferences*. 1997. Sacramento, CA.
- [20] Chen, Qiliang and Heydari, Babak. “The SoS Conductor: Orchestrating Resources with Iterative Agent-based Reinforcement Learning.” *Systems Engineering* Vol. 27 No. 4 (2024): pp. 715–727. DOI [10.1002/sys.21747](https://doi.org/10.1002/sys.21747).
- [21] Gadi, Vikranth, Topcu, Taylan G., Szajnarfarber, Zoe and Panchal, Jitesh H. “Heuristics for Solver-Aware Systems Architecting (SASA): A Reinforcement Learning Approach.” *Journal of Mechanical Design* Vol. 147 No. 2 (2024): p. 021704. DOI [10.1115/1.4066441](https://doi.org/10.1115/1.4066441).
- [22] Werfel, J. “Collective Construction With Robot Swarms.” *Morphogenetic Engineering*. Springer, Berlin, Heidelberg (2012): pp. 115–140.
- [23] Beckers, R., Holland, O. E. and Deneubourg, J. L. “From Local Actions to Global Tasks: Stigmergy and Collective Robotics.” *Prerational Intelligence*. Springer, Dordrecht (2000): pp. 1008–1022.
- [24] Dasgupta, P. “A Multiagent Swarming System for Distributed Automatic Target Recognition Using Unmanned Aerial Vehicles.” *IEEE Trans. Syst. Man Cybern. Part A Syst. Humans* Vol. 38 No. 3 (2008): pp. 549–563.
- [25] Ruini, F. and Cangelosi, A. “Extending the Evolutionary Robotics Approach to Flying Machines: An Application to MAV Teams.” *Neural Networks* Vol. 22 No. 5–6 (2009): pp. 812–821.
- [26] Lamont, G. B., Slear, J. N. and Melendez, K. “UAV Swarm Mission Planning and Routing Using Multi-Objective Evolutionary Algorithms.” *2007 IEEE Symposium on Computational Intelligence in Multi-Criteria Decision Making*: pp. 10–20. 2007. Honolulu, HI.
- [27] Wei, Y., Madey, G. R. and Blake, M. B. “Agent-Based Simulation for UAV Swarm Mission Planning and Execution.” *Proceedings of the Agent-Directed Simulation Symposium*: pp. 1–8. 2013.
- [28] Price, I. C. and Lamont, G. B. “GA Directed Self-Organized Search and Attack UAV Swarms.” *Proceedings of the 2006 Winter Simulation Conference*: pp. 1307–1315. 2006. IEEE, Monterey, CA.
- [29] Chen, Qiliang, Heydari, Babak and Moghaddam, Mohsen. “Leveraging Task Modularity in Reinforcement Learning for Adaptable Industry 4.0 Automation.” *Journal of Mechanical Design* Vol. 143 No. 7 (2021): p. 071701. DOI [10.1115/1.4049531](https://doi.org/10.1115/1.4049531).
- [30] Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA (2018).
- [31] Busoniu, L., Babuska, R. and De Schutter, B. “A Comprehensive Survey of Multiagent Reinforcement Learning.” *IEEE Trans. Syst. Man Cybern. Part C Appl. Rev.*, Vol. 38. 2: pp. 156–172. 2008.
- [32] Tampuu, A., Matiisen, T., Kodelja, D., Kuzovkin, I., Korjus, K., Aru, J., Aru, J. and Vicente, R. “Multiagent Cooperation and Competition With Deep Reinforcement Learning.” *PLoS One*, Vol. 12. 4: p. e0172395. 2017.
- [33] Foerster, J., Farquhar, G., Afouras, T., Nardelli, N. and Whiteson, S. “Counterfactual Multiagent Policy Gradients.” Vol. 32 No. 1 (2018).
- [34] Peng, X. B., Berseth, G., Yin, K. and Van De Panne, M. “Deeploco: Dynamic Locomotion Skills Using Hierarchical Deep Reinforcement Learning.” *ACM Trans. Graph.* Vol. 36 No. 4 (2017): pp. 1–13.
- [35] Zang, Yifan, He, Jinmin, Li, Kai, Fu, Haobo, Fu, Qiang, Xing, Junliang and Cheng, Jian. “Automatic grouping for efficient cooperative multi-agent reinforcement learning.” *Advances in Neural Information Processing Systems* Vol. 36 (2023): pp. 46105–46121.
- [36] Bukharin, Alexander, Li, Yan, Yu, Yue, Zhang, Qingru, Chen, Zhehui, Zuo, Simiao, Zhang, Chao, Zhang, Songan and Zhao, Tuo. “Robust multi-agent reinforcement learning via adversarial regularization: Theoretical foundation and stable algorithms.” *Advances in Neural Information Processing Systems* Vol. 36 (2023): pp. 68121–68133.
- [37] Chen, Yangkun, Yang, Kai, Tao, Jian and Lyu, Jiafei. “Novelty-Guided Data Reuse for Efficient and Diversified Multi-Agent Reinforcement Learning.” *arXiv preprint arXiv:2412.15517* (2024).
- [38] Tan, M. “Multiagent Reinforcement Learning: Independent vs. Cooperative Agents.” *Tenth International Conference on Machine Learning*: pp. 330–337. 1993. Amherst, MA.
- [39] Watkins, C. J. C. H. “Learning From Delayed Rewards.” Ph.D. Thesis, Cambridge University, Cambridge, UK. 1989.
- [40] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G. and Graves, A. et al. “Human-level Control Through Deep Reinforcement Learning.” *Nature* Vol. 518 No. 7540 (2015): pp. 529–533.
- [41] Foerster, J., Nardelli, N., Farquhar, G., Afouras, T., Torr, P. H., Kohli, P. and Whiteson, S. “Stabilising Experience Replay for Deep Multiagent Reinforcement Learning.” *International Conference on Machine Learning*: pp. 1146–1155. 2017. PMLR, Sydney, Australia.
- [42] Hochreiter, S. and Schmidhuber, J. “Long Short-Term Memory.” *Neural Comput.* Vol. 9 No. 8 (1997): pp. 1735–1780.
- [43] Chung, J., Gulcehre, C., Cho, K. and Bengio, Y. “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling.” *arXiv preprint* (2014)URL [1412.3555](https://arxiv.org/abs/1412.3555).
- [44] Hausknecht, M. and Stone, P. “Deep Recurrent Q-Learning for Partially Observable MDPs.” *arXiv preprint* (2015)URL [1507.06527](https://arxiv.org/abs/1507.06527).
- [45] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. and Mordatch, I. “Multiagent Actor-Critic for Mixed Cooperative-Competitive Environments.” *arXiv preprint* (2017)URL [1706.02275](https://arxiv.org/abs/1706.02275).

- [46] Brown, N. and Sandholm, T. “Superhuman AI for Multiplayer Poker.” *Science* Vol. 365 No. 6456 (2019): pp. 885–890.
- [47] Baker, B., Kanitscheider, I., Markov, T., Wu, Y., Powell, G., McGrew, B. and Mordatch, I. “Emergent Tool Use From Multiagent Autocurricula.” *arXiv preprint* (2019)URL [1909.07528](https://arxiv.org/abs/1909.07528).
- [48] Wu, S. A., Wang, R. E., Evans, J. A., Tenenbaum, J. B., Parkes, D. C. and Kleiman-Weiner, M. “Too Many Cooks: Bayesian Inference for Coordinating Multi-Agent Collaboration.” *Top. Cogn. Sci.* Vol. 13 No. 2 (2021): pp. 414–432.
- [49] Wang, Y. and De Silva, C. W. “Multi-Robot Box-Pushing: Single-Agent Q-Learning vs. Team Q-Learning.” *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*: pp. 3694–3699. 2006. IEEE, Beijing, China.
- [50] Rahimi, M., Gibb, S., Shen, Y. and La, H. M. “A Comparison of Various Approaches to Reinforcement Learning Algorithms for Multi-Robot Box Pushing.” *International Conference on Engineering Research and Applications*: pp. 16–30. 2018. Springer, Cham.
- [51] Foerster, J. N., Assael, Y. M., de Freitas, N. and Whiteson, S. “Learning to Communicate to Solve Riddles With Deep Distributed Recurrent Q-Networks.” *arXiv preprint* (2016)URL [1602.02672](https://arxiv.org/abs/1602.02672).
- [52] LaValle, S. M. *Planning Algorithms*. Cambridge University Press, New York (2006).
- [53] Jones, C. and Mataric, M. J. “Adaptive Division of Labor in Large-Scale Minimalist Multi-Robot Systems.” *Proceedings of the 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)*, Vol. 2: pp. 1969–1974. 2003. IEEE, Las Vegas, NV.
- [54] Groß, R., Bonani, M., Mondada, F. and Dorigo, M. “Autonomous Self-Assembly in Swarm-Bots.” *IEEE Trans. Rob.* Vol. 22 No. 6 (2006): pp. 1115–1130.
- [55] Qu, Zhongche, Zhang, Zhi and Liu, Cong. “Visual slam with 3d gaussian primitives and depth priors enabling novel view synthesis.” *2024 4th International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI)*: pp. 1–6. 2024. IEEE.
- [56] Liu, X. and Jin, Y. “Design of Transfer Reinforcement Learning Mechanisms for Autonomous Collision Avoidance.” *International Conference on Design Computing and Cognition*: pp. 303–319. 2018. Springer, Cham.